

User Manual for Running the Optimisation-based Scheduling Model

Madhu Sapkota

February 2025

1 Introduction

Surgical theatre scheduling and planning can be divided into three levels of decision-making:

- **Strategic Level:**

- Involves long-term decisions, such as capacity planning to maximise profit or cost efficiency.
- Often focuses on the *Case Mix Problem (CMP)*, which involves allocating theatres to different surgical specialities over an extended time horizon.

- **Tactical Level:**

- Involves medium-term planning, such as allocating theatre time slots to specialities and consultants/surgeons within a scheduling window.
- Known as the *Master Surgery Scheduling (MSS)* problem, which aims to:
 - * Maximise and balance resource utilisation.
 - * Address the overall requirements of specialities.

- **Operational Level:**

- Deals with short-term decisions, such as assigning specific surgical cases to theatres and planning associated resources.
- Referred to as the *Surgical Process Scheduling (SPS)* problem, which produces a detailed timeline for surgical cases, including:
 - * Start dates and times.
 - * Locations.
 - * Allocation of resources such as wards.

Scheduling approaches for tactical and operational planning can be broadly categorised into three types:

- **Block Scheduling:**

- Predefined time blocks are assigned to surgeons or consultants.
- Corresponding resources are blocked in advance.
- Typically follows a two-step approach:
 1. Solving the MSS problem by assigning time blocks to surgeons.
 2. Filling these blocks with specific surgical cases at the SPS level.

- **Modified Block Scheduling:**

- A variation of block scheduling that allows limited flexibility in block allocations.
- **Open Scheduling:**
 - A more flexible approach with no predefined session-to-surgeon assignments.
 - MSS and SPS problems are solved together.

2 Methodology Adopted

We adopt a block scheduling strategy at the operational level, where the number and duration of available operating room blocks—referred to as sessions (Figure 1)—are predefined. Each session corresponds to a specific time-block on a given day, reserved for a team of clinicians, including consultants, surgeons, anaesthetists, and nurses. Now, scheduling involves assigning patients from the waiting list to these allocated session-blocks considering multiple aspects like Trust’s objectives, patients’ satisfaction and resources available.

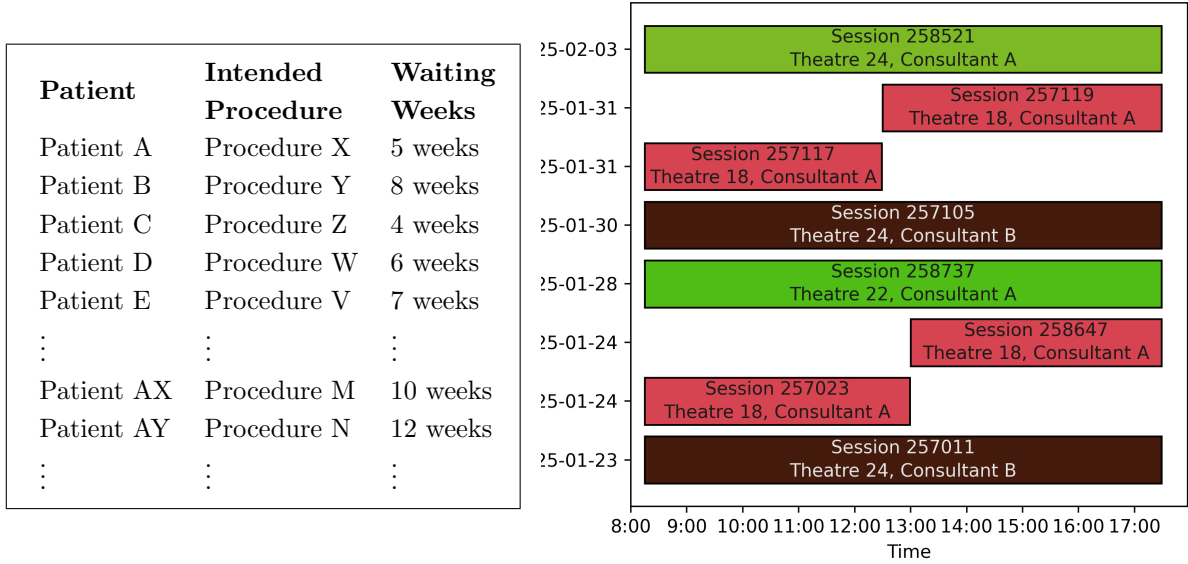


Figure 1: Patient pool from waiting list (left) and sessions’ time blocks reserved for theatres and consultants including their team (right) for different dates

The sequencing phase—i.e., the order in which patients undergo surgery within a session—is not considered, as the overall performance of the schedule is evaluated at the session level, making the internal arrangement of patients within a block irrelevant. Pre-trained machine learning models will be employed to estimate the duration of each surgery, providing predictions for every possible patient-session combination.

2.1 Problem Mathematical Representation

The scheduling problem will be treated as an optimisation problem, meaning the goal is to find the best possible solution by adjusting decision variables while meeting specific objectives and constraints.

2.1.1 Decision Variables

Let, P be the set of patients and S be the set of allocated sessions to be booked with patients. Solving with the block scheduling strategy means to determine whether a patient i is scheduled for any session-block j . For this a binary variable $x_{ij} \in \{0, 1\}$ is used indicating that i is assigned to session j ($x_{ij} = 1$) or not ($x_{ij} = 0$). The size of the variable is equal to the product of size of patients set and sessions set.

2.1.2 Objective Functions

This study at this phase considers elective surgery scheduling by addressing three key objectives:

1. Maximising theatre utilisation,
2. Minimising the probability of session overruns (or equivalently, maximising the likelihood of no overruns), and
3. Prioritising patients to achieve the maximum reduction in waiting times.

These objectives are defined based on the priorities of the hospital and patients, and are supported by related academic works, ensuring their practicality and relevance.

Objective 1-Theatre Utilisation (f_1): The first objective of the problem is to maximise the utilisation of available theatre time. It is represented mathematically as:

$$f_1(x) = \sum_{j \in S} \frac{1}{T_j} \sum_{i \in P} t_{ij} * x_{ij}$$

where,

- T_j : The total time available for session j .
- t_{ij} : Predicted surgery duration for the patient i when assessed in session j , obtained using machine learning models trained on historical data.
- γ_{ij} : theatre preparation time when patient i be assessed in session j .

Objective 2- Maximising Session no Overrun Probability (f_2) : Minimising the likelihood of session overrun can be alternatively expressed as maximising the probability of no overrun. Since surgical-time estimation relies on ML models, this probability is assessed based on the ML model's historical performance against real surgical durations. Detailed steps for computing the probability of no session overrun are provided in Appendix ??.

Objective 3- Patients' Waiting Time Objective (f_3) :

Reducing the RTT waiting times is the main concern in improving patient satisfaction and healthcare outcomes. To address this concern, the objective f_3 is designed in a way that the scheduling model contributes to reducing the backlog in an equitable manner, i.e., **prioritise patients with higher waiting times**, ensuring. Specifically, f_3 seeks to select patients who have waited the longest while also normalising their waiting times to maintain fairness across all cases.

$$f_3(x) = \frac{1}{\sum_{i \in P} WT_i} \sum_{i \in P} WT_i \left(\sum_{j \in S} x_{ij} \right),$$

where:

- WT_i is the waiting time of patient i ,
- $x_{ij} \in \{0, 1\}$ is a binary decision variable indicating whether patient i is assigned to session j ,
- P is the set of patients, and S is the set of sessions.

2.1.3 Multiple Objectives Assessment

The optimisation framework addresses the three objectives simultaneously, with the methodology differing based on the solution technique.

Multi-Objectives Approach In a multi-objective optimisation approach, each objective is treated independently, and trade-offs are evaluated at every iteration. The optimisation problem will be assessed as:

$$\max_x (f_1(x), f_2(x), f_3(x)) \quad (1)$$

Weighted Objectives Approach When multi-objective optimisation is impractical, the objectives are combined into a single scalar function using weighted coefficients. Then Objective 1 will be assessed as:

$$\max_x w_1 * f_1(x) + w_2 * f_2(x) + w_3 * f_3(x) \quad (2)$$

where w_1, w_2, w_3 are weights reflecting the relative importance of each objective.

2.1.4 Constraints

The optimisation process must adhere to several constraints while generating solution(s) that reflect operational requirements and also real-world resource limitations. For instance, condition (Equation 3) ensure the feasibility of the solution by guaranteeing that each patient is assigned to at most one session . Likewise, no session will be booked exceeding its allocated time (Equation 4, 5) considering the anticipated maximum utilisation (uti_{max}) of theatre and also considering turnover time (γ_{ij}).

$$\sum_{j \in S} x_{ij} \leq 1, \quad \forall i \in P \quad (3)$$

$$\sum_{i \in P} t_{ij} * x_{ij} \leq uti_{max} * T_j, \quad \forall j \in S \quad (4)$$

$$\sum_{i \in P} (t_{ij} + \delta_{ij}) x_{ij} \leq T_j, \quad \forall j \in S \quad (5)$$

2.2 Optimisation Solution Methods

We implemented two distinct optimisation methodologies: one based on Linear Programming (LP) and the other on an algorithmic metaheuristic approach.

2.2.1 SCIP Solver

The constraint integer programming solver SCIP (Achterberg 2009), was used to implement a Mixed Integer Programming (MIP) model, which is particularly effective for problems requiring mathematical rigour and exact solutions. A Python interface provider to SCIP, i.e., PySCIOpt (Maher et al. 2016) package is adopted so that code could be entirely written in Python and integrated with other Python-based subsidiary tool(s).

2.2.2 Simulated Annealing

Simulated Annealing (SA) represents a metaheuristic algorithmic approach designed to explore complex solution spaces efficiently. Unlike SCIP, which is limited by linearisation and computational intensity for large problems, SA is inherently flexible and scalable. It does not require the problem to be transformed into a linear form, making it well-suited for addressing multi-objective optimisation directly.

3 Model (python-codes) Running Steps

This section provides step-by-step instructions for users who want to run the scheduling model using the provided Python code. The model helps in scheduling surgical cases by considering above discussed constraints and objectives.

3.1 Packages Prerequisites

Before running the model, ensure you have the Python environment with necessary dependencies installed.

- pandas
- datetime
- numpy
- itertools
- pycipopt version preferred 5.1.1
- matplotlib
- random
- math
- sklearn version preferred 1.3.0
- scipy version preferred 1.11.1
- torch
- colorlover

3.2 Steps to Run the Scheduling Model

Note: Detail code with sample experiment result found in `Jupyter_notebook` file.

3.2.1 Import required function from built python modules

First ensure the followings own built modules are presented within the working directory:

- `Scheduling_Model`
- `data_processing_nd_encoding_related_functions`
- `predictive_model_related_functions`
- `procedure_time_and_variability_related_functions`
- `optimisation_related_functions` import

Then, import the following function:

```
from procedure_time_and_variability_related_functions  
load_timeslot_file
```

3.2.2 Ensure Input data requirements

- Input data files containing patient information and session plans.
 - `patient_df` : It holds patient list subset of overall waiting list. It should be in DataFrame format containing patient details like ID and Intended Procedure Code.
 - `sessions_plan`: A DataFrame outlining the available surgical sessions.
 - `patient_data` : It should hold overall features data about the patients required for prediction using trained ML.

- Required parameter values for surgical procedure time prediction. :

```
procedure_time_prediction_parameters = (models_types, data_model_categorisation,
pred_model_related_preprocessed_data, removed_data_from_processing,
model_input_features_all, possible_varying_data_for_patient, force_encoding_cols,
trained_models_direct)
```

Note: The surgical procedure time prediction related module can be referred for these data and parameters, and need to be reflected from the module.

3.2.3 Initialise the Scheduling Model

First, import the Scheduling_Model

```
from Scheduling_Model import Scheduling_Model
```

Then, instantiate the scheduling model by passing the required datasets:

```
sched_model = Scheduling_Model(patient_df, sessions_plan)
```

When initialised, `sched_model` will have various attributes, including but not limited to:

```
sched_model.CASES = tuple(patient_df["Patient ID"])
sched_model.CASES_RTT_WAIT_WEEKS = {}
sched_model.CASES_RTT_PRIORITY = {}
sched_model.SESIONS = tuple(sessions_plan["Session ID"])
sched_model.SESION_DURATION = pd.Series(sessions_plan["Total Slot Minutes"].values,
index=sessions_plan["Session ID"]).to_dict()
sched_model.SUM_ALL_SESSIONS_DURATION = sum(sessions_plan["Total Slot Minutes"])
sched_model.TASKS = tuple(product(patient_df["Patient ID"], sessions_plan["Session ID"]))
sched_model.TASKS_ASSIGNED = {}
```

These attributes help define the scheduling problem, tracking patient cases, session details, task assignments/solutions, and constraints.

3.2.4 Generate Task Durations

Generate estimated task durations based on patient data and procedure time prediction parameters for all tasks (case , session) combinations :

```
sched_model.generate_tasks_durations(patient_df, sessions_plan, patient_data,
procedure_time_prediction_parameters)
```

- `patient_data`: Additional patient-specific information.
- `procedure_time_prediction_parameters`: Parameters for predicting procedure durations.

This will assign the attribute `sched_model.TASKS_DURATION` to a dataframe with time duration data, where cases and session IDs are the row indices and columns, respectively.

3.2.5 Incorporate RTT Waiting Time (3rd Objective related data)

To account for cases with Referral-to-Treatment (RTT) waiting time.

```
sched_model.add_cases_RTT_waiting_time(patient_df)
```

```
sched_model.SUM_CASES_RTT_WAIT_WEEKS = sum(sched_model.CASES_RTT_WAIT_WEEKS.values())
```

This will assign the attribute `sched_model.CASES_RTT_WAIT_WEEKS` to a dictionary with patient ID as key, and waiting weeks as values. This data assist that patients waiting the longest are prioritised.

3.2.6 Generate and Store Task Duration Deviation Probabilities (2^{nd} Objective requirement)

This step is only required if 2^{nd} objective is being considered.

```
sched_model.generate_chances_for_tasks_durations(patient_df, sessions_plan, 30,
patient_data, procedure_time_prediction_parameters)
```

It generates task duration deviation chances (ratios) based on 30 (can be different) data samples from the past and assigns the attribute `sched_model.TASKS_DURATION_DEVIATION_RATIO_CHANCES` to a dictionary with key (patient ID, Session ID) as key, and list of ratio chances as values.

Next, obtain the mean and standard deviation for these deviation ratios.

```
sched_model.obtain_mean_sd_for_tasks_durations_chances(patient_df, sessions_plan,
30, patient_data, procedure_time_prediction_parameters)
```

It assigns the attribute `sched_model.TASKS_DURATION_DEVIATION_RATIO_CHANCES_MEAN_SD` to a dictionary with key (patient ID, Session ID) as key, and list of 2 values, mean and s.d. of

```
sched_model.TASKS_DURATION_DEVIATION_RATIO_CHANCES_MEAN_SD
```

3.2.7 Set the theatre prep min required in between consecutive procedures and maximum session booking ratio constraint

```
sched_model.session_max_util = ... (default 0.9)
sched_model..theatre_prpn_min = ... (in min, default 7.5)
```

3.2.8 Run the Optimisation Algorithm

Two different algorithms (Section 2.2) can be chosen to create an optimised schedule:

Option 1: MIP Solver

```
hyper_param SCIP = {'time limit': len(patient_df)*len(sessions_plan)/20}
```

```
obj_weightage = {'objective1':0.7, 'objective2':0.2, 'objective3':0.1}
```

```
sched_model.create_schedule_with_optimisation(optimisation_algorithm = 'MIP Solver',
objectives = ('objective1', 'objective2', 'objective3'),
weightage= obj_weightage, hyper_param= hyper_param SCIP)
```

- `hyper_param SCIP`: Defines the time limit for the solver based on dataset size.
- `objectives`: Tuple of objectives to be considered in the optimisation process.
- `obj_weightage`: Specifies the importance of each objective in the optimisation process.
- `optimisation_algorithm`: The algorithm used to solve the scheduling problem (MIP Solver in this case).

It assigns the attributes:

```
sched_model.schedule_new,
sched_model.sessions_updated,
sched_model.patients_not_scheduled,
sched_model.best_solution
sched_model.scip_solver
```

Option 2: Simulated Annealing

```
hyper_para_sim = {'INITIAL_TEMPERATURE': 1000000 , 'FINAL_TEMPERATURE' : 0.001,
                  'ALPHA' : 0.999, 'MAX_ITERATIONS' : 100000}
obj_weightage = {'objective1':0.75, 'objective2':0.25}

start_time = time.time()

sched_model.create_schedule_with_optimisation(optimisation_algorithm = 'Simulated Annealing',
objectives = ('objective1', 'objective2', 'objective3'), weightage= obj_weightage,
hyper_param= hyper_para_sim)

end_time = time.time() total_time = end_time - start_time
print("Total time:", total_time, "seconds")
```

- **hyper_para_sim**: Defines parameters for the Simulated Annealing process, such as temperature decay.
- **obj_weightage**: Specifies the importance of each objective in the optimisation process.
- **optimisation_algorithm**: Uses Simulated Annealing as the optimisation method.
- The execution time is measured for performance evaluation.

It assigns the attributes such as

```
sched_model.schedule_new,
sched_model.sessions_updated,
sched_model.patients_not_scheduled,
sched_model.best_solution
sched_model.best_kpis_list
```

3.3 Model Result Testing

After generating the schedule, evaluate the optimisation results using multi-objective performance testing:

```
from optimisation_related_functions import multi_objectives

multi_objectives(sched_model, None, 0, cases_to_skip = sched_model.cases_not_considered,
objectives_list = ('objective1', 'objective2', 'objective3'), prep_min= sched_model.theatre_prep_min,
cdf_table_data = z_cdf_data, individual = sched_model.best_solution.T.flatten())
```

- This function tests the model's performance based on the defined objectives.
- **prep_min**: Preparation/Turnover time considered in between consecutive procedures within a session.
- **cdf_table_data**: Table of cumulative distribution functions for the evaluation of probability.

This will return the objective values as tuples with the length as similar to the **objectives_list**.

3.4 Schedule Visualisation

To visualize theatre occupancy and assess resource allocation:

```
from procedure_time_and_variability_related_functions import plot_theatre_occupancy_data

with open('label_mapping.pickle', 'rb') as file:
    label_mapping = pickle.load(file)

ax2= plot_theatre_occupancy_data(sched_model.schedule_new[['Scheduled Start Date/Time',
'Planned H4 Minutes', 'Patient ID', 'Intended Procedure Code', 'Session ID']],
(5.5,len(sessions_plan)/1.5), label_mapping, color_mapping_col = 'Intended Procedure Code',
grouping_col='Session ID' )
```

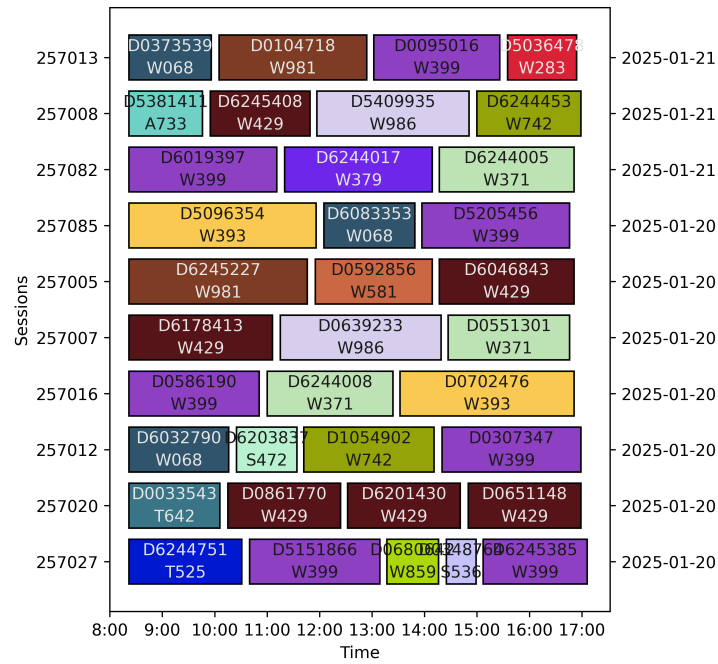


Figure 2: Visualisation of a generated Schedule from a sample experiment

- This function generates a graphical representation of theatre occupancy.
- The x-axis represents time, while different colors represent procedures.
- `color_mapping_col`: Maps different procedures to specific colors.
- `grouping_col`: Groups procedures by session ID for better visualisation.

3.5 Notes

- Ensure all input datasets are correctly formatted before running the model.
- Review updated session plans to ensure optimal scheduling.

3.6 Troubleshooting

If you encounter issues:

- Check that all required parameters are properly defined.
- Verify that input DataFrames (`patient_df`, `sessions_plan`) contain the expected columns.
- Ensure your Python environment includes all necessary dependencies.

For further assistance, contact the development team or refer to the model documentation.

References

- Achterberg, T. (2009), ‘Scip: solving constraint integer programs’, *Mathematical Programming Computation* **1**, 1–41.
- Maher, S., Miltenberger, M., Pedroso, J. P., Rehfeldt, D., Schwarz, R. & Serrano, F. (2016), PySCIPOpt: Mathematical programming in python with the SCIP optimization suite, *in* ‘Mathematical Software – ICMS 2016’, Springer International Publishing, pp. 301–307.